

Practical No: - 5

ON

Cryptography and Network Security - SSCS 4020

TITLE: Demonstration of DES Encryption using Python

BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

Problem Statement – 1

To implement the DES (Data Encryption Standard) algorithm for encrypting and decrypting a plaintext message using Python.

Theory

DES (Data Encryption Standard) is a symmetric-key block cipher used for encrypting digital data. It uses the same secret key for both encryption and decryption. DES operates on **64-bit (8-byte)** blocks of data and uses an effective **56-bit key** (stored as an 8-byte key with parity bits). Since DES works with fixed-size blocks, padding is added whenever the plaintext size is not a multiple of 8 bytes.

Although DES has been replaced by more secure algorithms like AES, it is still widely studied for understanding the fundamentals of symmetric encryption.

INPUT :

!pip install pycryptodome

```
Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
    Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
      2.3/2.3 MB 25.4 MB/s eta 0:00:00
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0
```

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

# Secret key (8 bytes)
key = b'ABCDEFGH'

# Create DES cipher
cipher = DES.new(key, DES.MODE_ECB)

# Input plaintext
plaintext = input("Enter Plaintext: ")

# Convert into bytes
data = plaintext.encode()

# Apply padding
padded_data = pad(data, DES.block_size)

# Encrypt
ciphertext = cipher.encrypt(padded_data)

print("\nEncrypted Ciphertext (Bytes):")
print(ciphertext)
```

```
print("\nEncrypted Ciphertext (Hex):")
print(ciphertext.hex())

# Create DES cipher for decryption
decipher = DES.new(key, DES.MODE_ECB)

# Decrypt
decrypted = decipher.decrypt(ciphertext)

# Remove padding
original = unpad(decrypted, DES.block_size)

print("\nDecrypted Text:", original.decode())
```

OUTPUT:

```
Enter Plaintext: Network Security

Encrypted Ciphertext (Bytes):
b'\xd7Lc_\x17\xe5\xb5,\xd3]Nc\xe5":\x05_;\xc8;\x9d\xf8\xee'

Encrypted Ciphertext (Hex):
d74c635f170e35b52cd35d4e63e5223a055f3bc83b9df8ee

Decrypted Text: Network Security
```

Problem Statement – 2

Aim: To simulate secure communication using DES encryption with a shared secret key.

Theory

In secure communication, both the sender and the receiver share the same secret key. The sender encrypts the confidential message using DES before transmission. The receiver decrypts the received ciphertext using the same secret key and retrieves the original plaintext. Without the correct key, the encrypted message cannot be decrypted correctly.

INPUT :

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

# Shared Secret Key
key = b'ABCDEFGH'

print(" Sender :")
```

```
message = input("Enter Message :→")
cipher = DES.new(key, DES.MODE_ECB)
ciphertext = cipher.encrypt(
    pad(message.encode(), DES.block_size)
)
print("\nCiphertext Sent (Hex):")
print(ciphertext.hex())
print("\n Receiver :→")
decipher = DES.new(key, DES.MODE_ECB)
received = unpad(
    decipher.decrypt(ciphertext),
    DES.block_size
)
print("Received Ciphertext :")
print(ciphertext.hex())
print("Decrypted Message :")
print(received.decode())
```

OUTPUT:

```
Sender :
Enter Message :→Cryptography & Network Security

Ciphertext Sent (Hex):
ec66c99f3148a52f1ec1c0e65c0eb3f89f2138deb0d03a12d5d82cbdd11e4931

Receiver :→
Received Ciphertext :
ec66c99f3148a52f1ec1c0e65c0eb3f89f2138deb0d03a12d5d82cbdd11e4931
Decrypted Message :
Cryptography & Network Security
```